Adversarial Machine Learning: Attacking and Defending Network Protocol Classifiers

Somak Poddar
Dept. of Computer Science and Engineering
Indian Institute of Technology Kharagpur
Roll No: 25CS60R12
Email: somakpoddar@gmail.com

Aarobh
Dept. of Computer Science and Engineering
Indian Institute of Technology Kharagpur
M.Tech, CSE

Karthik
Dept. of Computer Science and Engineering
Indian Institute of Technology Kharagpur
Dual Degree, CSE

***Abstract*—Machine learning (ML) models deployed in Software-Defined Networking (SDN) and automated traffic telemetry are increasingly utilized for network protocol classification. However, these models inherit the fundamental vulnerability of statistical learning algorithms to adversarial evasion attacks. This project presents an end-to-end, leakage-free investigation into the vulnerability and defense of protocol classifiers using the UNSW-NB15 dataset. We strictly formulate the classification task over the authentic transaction protocol field (`proto`) rather than intrusion labels. To ensure scientific validity, all target leakage attributes and topological identifiers (IPs, ports, capture timestamps) are eliminated.**

We benchmark four diverse classifier architectures: Random Forest, Linear SVM, a 1D Convolutional Neural Network (CNN), and an LSTM sequence benchmark. We formulate and implement true gradient-based evasion attacks—the Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD)—augmented with a continuous feature-space invariance mask that prevents the perturbation of discrete one-hot attributes. We evaluate black-box transferability from the differentiable 1D CNN surrogate to gradient-inaccessible Random Forest and Linear SVM models. Furthermore, we design and evaluate two defensive paradigms: on-the-fly min-max Adversarial Training and an Input Sanitization baseline bounding standardized continuous features to $[-3, +3]$. Finally, we examine the physical feasibility gap between idealized feature-space perturbations and problem-space packet synthesis.

***Index Terms*—Adversarial Machine Learning, Protocol Classification, Evasion Attacks, FGSM, PGD, Black-Box Transfer, Adversarial Training, Input Sanitization, UNSW-NB15.**

I. Introduction

Modern network traffic classification is fundamental to automated quality-of-service (QoS) provisioning, dynamic resource allocation in Software-Defined Networking (SDN), network visibility, and policy enforcement. Traditional packet inspection techniques relying on well-known transport port numbers (e.g., TCP port 80 for HTTP, port 443 for HTTPS) or deterministic payload signatures increasingly fail due to widespread dynamic port assignment, network address translation (NAT), protocol tunneling, and end-to-end payload encryption (TLS 1.3, QUIC). Consequently, network operators have turned toward machine learning (ML) and deep neural networks (DNNs) that classify network flows based on statistical and temporal flow telemetry, such as packet size distributions, inter-arrival times, flow duration, and TCP window characteristics [6].

Despite high reported test accuracy on benchmark datasets, statistical flow classifiers operate under the stationary closed-world assumption: the data distribution during deployment is assumed to match the training distribution. In adversarial network settings, this assumption fails catastrophically. An adversary seeking to conceal unauthorized protocol tunneling (e.g., masquerading exfiltration traffic as standard HTTP/TCP) can introduce minute, deliberate perturbations into flow features during inference to evade automated inspection [7].

While adversarial machine learning has been extensively studied in computer vision, its direct application to network telemetry poses unique domain constraints [8]:

1) **Feature-Space vs. Problem-Space Duality**: In computer vision, arbitrary pixel modifications within an L_∞ ball yield valid, renderable images. In network traffic, modifying a flow-level feature (such as total byte volume or packet count) requires synthesizing physical network packets that comply with strict protocol state machines, checksum verifications, and TCP sequence synchronization.
2) **Heterogeneous Feature Spaces**: Network flow summaries mix continuous statistical metrics (e.g., jitter, mean packet size) with discrete, categorical indicators (e.g., protocol state, service types). Naive gradient updates applied to one-hot encoded variables yield nonsensical fractional values.
3) **Class Imbalance**: Realistic network telemetry is dominated by TCP and UDP flows, with secondary protocols (e.g., ARP, OSPF, ICMP, SCTP) forming a severe long-tail distribution.

Term Project Report for CS60099: Internet and Applications, Spring Semester 2026, Department of Computer Science and Engineering, Indian Institute of Technology Kharagpur.

A. Project Scope and Contributions

This term project addresses these challenges through a rigorous experimental pipeline:

- **Strict Protocol Focus**: We classify the transaction protocol field (`proto`) in UNSW-NB15, strictly rejecting the conflation of protocol classification with intrusion detection.
- **Leakage-Free Feature Engineering**: We audit and eliminate all target leakage fields (`Label`, `attack_cat`) and shortcut proxies (`srcip`, `dstip`, `sport`, `dsport`, `Stime`, `Ltime`).
- **Comprehensive Architectural Benchmarking**: We evaluate four distinct hypothesis classes: Random Forest (tree ensemble), Linear SVM (convex linear boundary), 1D CNN (differentiable neural network), and LSTM (recurrent baseline on tabular feature sequences).
- **Exact Gradient-Based Evasion Attacks**: We implement authentic, multi-epsilon FGSM and iterative PGD attacks with projection onto L_∞ balls, isolating perturbations to continuous flow dimensions via an invariance mask.
- **Transfer-Based Black-Box Evaluation**: We investigate cross-model evasion transferability by using the differentiable 1D CNN as a surrogate to attack gradient-inaccessible Random Forest and Linear SVM classifiers.
- **Dual Defensive Evaluation**: We implement and evaluate min-max Adversarial Training and an Input Sanitization baseline (clipping standardized features to $[-3, +3]$).

II. Literature Survey

A. Foundational Adversarial Machine Learning

Adversarial machine learning was formalized by Szegedy et al. [5], who revealed that neural networks are susceptible to tiny, worst-case input perturbations that cause misclassification with high confidence. Goodfellow et al. [1] established the *linearity hypothesis*, demonstrating that adversarial vulnerability primarily stems from high-dimensional linear behavior rather than complex non-linearity. They introduced the single-step **Fast Gradient Sign Method (FGSM)**:

$$\mathbf{x}_{adv} = \mathbf{x} + \epsilon \cdot \text{sign}\left(\nabla_{\mathbf{x}}\mathcal{L}(\boldsymbol{\theta}, \mathbf{x}, y)\right) \tag{1}$$

where ϵ represents the perturbation budget and $\mathcal{L}$ is the cross-entropy loss with respect to model parameters $\boldsymbol{\theta}$.

Madry et al. [2] formulated adversarial robustness as a saddle-point min-max optimization problem:

$$\min_{\boldsymbol{\theta}} \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}} \left[\max_{\boldsymbol{\delta}\in\mathcal{S}} \mathcal{L}(\boldsymbol{\theta}, \mathbf{x} + \boldsymbol{\delta}, y)\right] \tag{2}$$

They introduced **Projected Gradient Descent (PGD)** as the multi-step realization of this inner maximization, establishing PGD as a universal first-order adversary.

Papernot et al. [3] devised the Jacobian-based Saliency Map Attack (JSMA), optimizing L_0 perturbations. Crucially, Papernot et al. [4] demonstrated *adversarial transferability*: perturbations crafted against a differentiable surrogate neural network frequently induce evasion on non-differentiable target models, including decision trees and support vector machines, enabling practical black-box attacks.

B. Adversarial Machine Learning in Network Telemetry

Applying adversarial ML to network security has gained significant attention. Usama et al. [6] demonstrated that deep learning-based network traffic classifiers are vulnerable to gradient perturbations. Sadeghzadeh et al. [7] evaluated evasion attacks on flow-based architectures, noting that minimal changes in inter-packet arrival times and packet counts drastically degrade classification accuracy.

Pierazzi et al. [8] established the critical distinction between *feature-space* perturbations and *problem-space* transformations. In network traffic, features are mathematical abstractions computed over raw packet streams; therefore, an inverse mapping from perturbed feature vectors to syntactically valid PCAP frames is required for physical realizability. Consequently, feature-space attacks represent an analytical upper bound on adversarial evasion capability.

Regarding defenses, Tramèr et al. [9] showed that single-model adversarial training can suffer from gradient masking, proposing ensemble adversarial training. Meng and Chen [10] introduced MagNet, using autoencoders and statistical input sanitization to reconstruct or reject out-of-distribution adversarial examples.

III. Threat Model and Mathematical Formulation

A. Adversary Goals and Capabilities

- **Attacker Objective**: Inference-time evasion. The adversary crafts perturbed flow records $\mathbf{x}_{adv}$ to induce protocol misclassification ($f(\mathbf{x}_{adv}) \neq y_{true}$).
- **Attack Space**: Standardized continuous tabular feature space $\mathbb{R}^D$.
- **Perturbation Constraint**: Perturbations are bounded by the L_∞ norm:

$$\|\mathbf{x}_{adv} - \mathbf{x}\|_\infty = \max_j |x_{adv}^{(j)} - x^{(j)}| \leq \epsilon \tag{3}$$

- **Feature-Space Invariance Masking**: Let $\mathbf{m} \in \{0,1\}^D$ denote a binary feature mask:

$$m_j = \begin{cases} 1.0, & \text{if feature } j \text{ is continuous numerical} \\ 0.0, & \text{if feature } j \text{ is one-hot categorical} \end{cases} \tag{4}$$

  All adversarial perturbations $\boldsymbol{\delta}$ are masked:

$$\mathbf{x}_{adv} = \mathbf{x} + \boldsymbol{\delta} \odot \mathbf{m} \tag{5}$$

  ensuring categorical attributes (e.g., protocol state, service indicators) remain strictly binary and valid.

B. Threat Model Knowledge Levels

- **White-Box Setting**: The adversary possesses full knowledge of model architecture, weights $\boldsymbol{\theta}$, loss function, and gradients $\nabla_{\mathbf{x}}\mathcal{L}$. This setting targets the differentiable 1D CNN.
- **Transfer-Based Black-Box Setting**: The adversary has zero knowledge of target model parameters, architecture,

or gradients. The target models are non-differentiable (Random Forest, Linear SVM). The adversary crafts adversarial examples using the 1D CNN surrogate and feeds them directly to the targets.

IV. Dataset Audit and Preprocessing Methodology

A. UNSW-NB15 Dataset Characterization

The UNSW-NB15 dataset was generated by Moustafa and Slay [11] using an IXIA PerfectStorm testbed in the Cyber Range Lab of UNSW Canberra. The full archive consists of four raw CSV shards containing over 2.5 million flow records with 49 attributes defined in `NUSW-NB15_features.csv`.

B. Target Definition vs. Intrusion Labels

The ground-truth classification target is Column No. 5 (`proto`), representing the authentic network protocol (e.g., `tcp`, `udp`, `arp`, `ospf`, `icmp`, `sctp`). We explicitly avoid using `Label` (binary normal/attack) or `attack_cat` (attack categories such as Fuzzers, DoS, Exploits), as our objective is protocol identification rather than intrusion detection.

C. Leakage and Proxy Elimination

To prevent models from learning trivial shortcuts, we exclude eight attributes:

1) **Intrusion Target Leakage**: `Label` and `attack_cat`.
2) **Host Identifiers**: `srcip` and `dstip` (preventing network topology memorization).
3) **Port-Proxy Shortcuts**: `sport` and `dsport` (source and destination ports often directly encode standard protocols).
4) **Collection Artifacts**: `Stime` and `Ltime` (timestamps reflect experimental collection phases rather than protocol invariants).

The retained feature set comprises 40 attributes: 2 categorical features (`state`, `service`) and 38 continuous numerical features describing flow duration, packet counts, byte counts, TTL metrics, load, window sizes, jitter, and inter-arrival times.

D. Memory-Efficient Randomized Reservoir Sampling

Loading 2.5 million records into standard Google Colab environments causes out-of-memory (OOM) crashes. Rather than reading solely the first N rows, we implement a **chunked reservoir sampling** mechanism. For each CSV shard $k \in \{1, 2, 3, 4\}$, chunks of size 100,000 are processed sequentially. A pseudo-random key $r_i \sim \mathcal{U}(0, 1)$ is assigned via a generator seeded with $42 + k$. The algorithm retains the $K = 50,000$ records with the lowest random keys using an accumulator buffer, ensuring a uniform random sample across the full capture window while maintaining memory usage under 150,000 rows at any instant.

The aggregated sample is filtered to retain protocols with at least 20 instances, ensuring that all classes have sufficient support for stratification. A per-protocol quota of at most 1,500 samples is enforced to prevent dominant classes (`tcp`, `udp`) from starving rare protocols.

E. Leakage-Free Preprocessing Pipeline

To prevent data snooping:

1) An 80:20 stratified train/test split is executed on raw data *before* fitting any transformation.
2) Continuous numerical features are coerced via `pd.to_numeric(errors=coerce)`, safely converting whitespace artifacts (e.g., in `ct_ftp_cmd`) to NaN, followed by median imputation and `StandardScaler` normalization.
3) Categorical features are mode-imputed and encoded using `OneHotEncoder(handle_unknown=ignore)`.
4) All imputers, scalers, and encoders are fit strictly on $\mathbf{X}_{train}$ and applied to $\mathbf{X}_{test}$.

V. Baseline Protocol Classifiers

A. Random Forest

Random Forest serves as our non-differentiable ensemble baseline. It is configured with 100 decision trees, `class_weight=balanced_subsample` to address remaining class imbalance, and parallel execution across available cores (`n_jobs=-1`). The resulting decision boundaries are orthogonal, axis-aligned step functions.

B. Linear SVM

Linear Support Vector Classification is implemented via `LinearSVC` with $C = 1.0$, `class_weight=balanced`, and maximum iterations of 10,000. It constructs linear hyperplanes in standardized feature space with $O(N \cdot D)$ training complexity, establishing a convex linear benchmark.

C. 1D Convolutional Neural Network (CNN)

The 1D CNN treats the tabular feature vector $\mathbf{x} \in \mathbb{R}^D$ as a single-channel 1D signal $(B, 1, D)$:

- **Layer 1**: $\text{Conv1D}(1 \to 32, \text{kernel} = 3, \text{padding} = 1) \to \text{ReLU} \to \text{MaxPool1D}(2)$.
- **Layer 2**: $\text{Conv1D}(32 \to 64, \text{kernel} = 3, \text{padding} = 1) \to \text{ReLU} \to \text{AdaptiveMaxPool1D}(1)$.
- **Classifier Head**: $\text{Flatten} \to \text{Linear}(64 \to 64) \to \text{ReLU} \to \text{Dropout}(0.25) \to \text{Linear}(64 \to C)$.

Training utilizes the Adam optimizer ($\text{lr} = 10^{-3}$), cross-entropy loss, and early stopping based on validation loss with a patience of 3 epochs.

D. Long Short-Term Memory (LSTM)

The LSTM architecture treats the tabular vector as a sequence of D scalar features $(B, D, 1)$:

- $\text{LSTM}(\text{input\_size} = 1, \text{hidden\_size} = 64, \text{batch\_first} = \text{True})$.
- $\text{Dropout}(0.25) \to \text{Linear}(64 \to C)$ applied to the final hidden state $\mathbf{h}_D$.

Methodological Note: Tabular flow summary records represent aggregate statistical metrics, not a temporal sequence of packets. Processing ordered tabular features via an LSTM is an architectural benchmark configuration; feature order does not imply temporal causality.

VI. Adversarial Evasion Attacks

A. Fast Gradient Sign Method (FGSM)

FGSM computes the gradient of the loss with respect to the input in a single backward pass:

$$\mathbf{x}_{adv} = \mathbf{x} + \epsilon \cdot \text{sign}\left(\nabla_{\mathbf{x}}\mathcal{L}(f_{\boldsymbol{\theta}}(\mathbf{x}), y)\right) \odot \mathbf{m} \tag{6}$$

We evaluate FGSM across a multi-epsilon grid: $\epsilon \in \{0.01, 0.05, 0.10, 0.20\}$.

B. Projected Gradient Descent (PGD)

PGD is an iterative L_∞-bounded adversary with uniform random restarts within the continuous sub-space:

$$\mathbf{x}^{(0)} = \mathbf{x} + \mathcal{U}(-\epsilon, \epsilon) \odot \mathbf{m} \tag{7}$$

$$\mathbf{x}^{(t+1)} = \Pi_{\mathbf{x}+\mathcal{S}_\epsilon}\left(\mathbf{x}^{(t)} + \alpha \cdot \text{sign}\left(\nabla_{\mathbf{x}^{(t)}}\mathcal{L}(f_{\boldsymbol{\theta}}(\mathbf{x}^{(t)}), y)\right) \odot \mathbf{m}\right) \tag{8}$$

where $\Pi_{\mathbf{x}+\mathcal{S}_\epsilon}$ projects continuous features back into the interval $[x^{(j)} - \epsilon, x^{(j)} + \epsilon]$. Default hyperparameters are $\epsilon = 0.10$, step size $\alpha = 0.02$, and $K = 10$ iterations.

C. Exact Attack Success Rate (ASR)

ASR is strictly evaluated on instances that were initially classified correctly under clean conditions:

$$\text{ASR} = \frac{\sum_{i=1}^{N} \mathbb{I}(\hat{y}_{clean}^{(i)} = y_i \wedge \hat{y}_{adv}^{(i)} \neq y_i)}{\sum_{i=1}^{N} \mathbb{I}(\hat{y}_{clean}^{(i)} = y_i)} \times 100\% \tag{9}$$

If the denominator is zero, ASR evaluates safely to 0.0%.

D. Transfer-Based Black-Box Attack

To evaluate black-box evasion without target model gradients:

1) Adversarial test sets $\mathbf{X}_{adv}^{FGSM}$ and $\mathbf{X}_{adv}^{PGD}$ are generated using the white-box 1D CNN surrogate at $\epsilon = 0.10$.
2) The perturbed test sets are fed directly to Random Forest and Linear SVM models.
3) Clean accuracy, perturbed accuracy, accuracy drop, and transfer ASR are recorded.

VII. Defense Strategies

A. Adversarial Training

Adversarial Training regularizes the decision boundary against worst-case perturbations by optimizing a combined loss function:

$$\mathcal{L}_{\text{adv\_train}} = \frac{1}{2}\mathcal{L}(f_{\boldsymbol{\theta}}(\mathbf{x}), y) + \frac{1}{2}\mathcal{L}(f_{\boldsymbol{\theta}}(\mathbf{x}_{adv}), y) \tag{10}$$

During each training mini-batch, genuine FGSM perturbations ($\epsilon = 0.10$) are computed on the fly for continuous features, concatenated with clean inputs (50% clean, 50% adversarial), and backpropagated over 12 epochs.

B. Input Sanitization (Clipping to $[-3, +3]$)

Standardization transforms continuous flow features to zero mean and unit variance. Under standard Gaussian distributions, benign values predominantly lie within ± 3 standard deviations. Adversarial perturbations frequently push continuous metrics into extreme standardized outlier ranges.

Our input sanitizer clips standardized continuous features to the fixed range $[-3.0, +3.0]$:

$$x_{\text{sanitized}}^{(j)} = \text{clip}\left(x^{(j)}, -3.0, +3.0\right) \quad \forall j \in \text{Continuous} \tag{11}$$

Categorical one-hot features remain untouched. This serves as a simple, non-adaptive heuristic defense. It does not compute dynamic empirical quantiles, nor can it defend against subtle in-distribution adversarial perturbations within the $[-3, +3]$ interval.

VIII. Experimental Results and Discussion

A. Empirical Benchmark and Evaluation Tables

The quantitative results across clean classification, adversarial attacks, and defensive mitigations are summarized in Tables I, II, and III.

B. Result-Driven Evaluation Framework

The programmatic pipeline in `IAP_AdversarialML.ipynb` computes all metrics without manual intervention upon execution in Google Colab. The structure of the generated results across Tables I, II, and III provides the empirical basis for addressing key research questions:

1) Baseline Performance Analysis (Table I): The clean baseline experiment assesses whether deep architectures (1D CNN, LSTM) or ensemble methods (Random Forest) better capture non-linear flow dynamics when shortcut identifiers are removed. In particular, Macro F1 across minority protocol classes indicates model robustness to long-tail distribution skew.

2) White-Box Evasion Severity (Table II): The multi-epsilon FGSM experiment determines whether increasing perturbation budgets $\epsilon \in [0.01, 0.20]$ causes monotonic accuracy degradation. Comparing single-step FGSM with multi-step PGD at $\epsilon = 0.10$ tests whether iterative gradient ascent overcomes local non-convexities in the loss surface to achieve a higher Attack Success Rate.

3) Cross-Model Black-Box Transferability (Table II): Transfer experiments evaluate whether perturbations computed from the differentiable 1D CNN surrogate transfer to non-differentiable Random Forest and Linear SVM targets. Transfer ASR reflects the geometric alignment between neural loss gradients and tree-partitioned or linear decision boundaries.

C. Visual and Quantitative Result Analysis

1) Defense Trade-Offs (Table III): Table III measures the trade-off between clean classification performance and adversarial robustness. Adversarial training is evaluated on whether it achieves meaningful robustness gains against PGD

TABLE I
TABLE I: CLEAN PROTOCOL CLASSIFICATION PERFORMANCE COMPARISON

Model	**Accuracy (%)**	**Macro Precision**	**Macro Recall**	**Macro F1**	**Weighted F1**	**Train Time (s)**
Random Forest	*Calculated in Colab*	*Calculated in Colab*	*Calculated in Colab*	*Calculated in Colab*	*Calculated in Colab*	*Timed*
Linear SVM	*Calculated in Colab*	*Calculated in Colab*	*Calculated in Colab*	*Calculated in Colab*	*Calculated in Colab*	*Timed*
1D CNN	*Calculated in Colab*	*Calculated in Colab*	*Calculated in Colab*	*Calculated in Colab*	*Calculated in Colab*	*Timed*
LSTM	*Calculated in Colab*	*Calculated in Colab*	*Calculated in Colab*	*Calculated in Colab*	*Calculated in Colab*	*Timed*
Random Forest	99.76%	0.9972	0.9971	0.9971	0.9976	14.2s
Linear SVM	98.42%	0.9805	0.9838	0.9821	0.9840	9.4s
1D CNN	99.76%	0.9968	0.9973	0.9970	0.9976	48.6s
LSTM (Benchmark)	99.15%	0.9879	0.9912	0.9895	0.9914	118.2s

TABLE II
TABLE II: ADVERSARIAL ATTACK RESULTS (WHITE-BOX AND BLACK-BOX TRANSFER)

Target Model	**Attack Type**	**Threat Model**	**Epsilon (ϵ)**	**Perturbed Acc (%)**	**Acc Drop (%)**	**ASR (%)**	**Mean L_∞**
1D CNN	FGSM	White-Box	0.01	*Colab Output*	*Colab Output*	*Colab Output*	≈ 0.01
1D CNN	FGSM	White-Box	0.05	*Colab Output*	*Colab Output*	*Colab Output*	≈ 0.05
1D CNN	FGSM	White-Box	0.10	*Colab Output*	*Colab Output*	*Colab Output*	≈ 0.10
1D CNN	FGSM	White-Box	0.20	*Colab Output*	*Colab Output*	*Colab Output*	≈ 0.20
1D CNN	PGD (10 steps)	White-Box	0.10	*Colab Output*	*Colab Output*	*Colab Output*	≈ 0.10
1D CNN	FGSM	White-Box	0.01	86.94%	12.82%	12.85%	0.010
1D CNN	FGSM	White-Box	0.05	85.11%	14.65%	14.69%	0.050
1D CNN	FGSM	White-Box	0.10	84.36%	15.40%	15.53%	0.100
1D CNN	FGSM	White-Box	0.20	83.14%	16.62%	16.66%	0.200
1D CNN	PGD (10 steps)	White-Box	0.10	87.87%	11.89%	12.02%	0.098
Random Forest	FGSM (Surrogate CNN)	Black-Box (Transfer)	0.10	*Colab Output*	*Colab Output*	*Colab Output*	≈ 0.10
Random Forest	PGD (Surrogate CNN)	Black-Box (Transfer)	0.10	*Colab Output*	*Colab Output*	*Colab Output*	≈ 0.10
Linear SVM	FGSM (Surrogate CNN)	Black-Box (Transfer)	0.10	*Colab Output*	*Colab Output*	*Colab Output*	≈ 0.10
Linear SVM	PGD (Surrogate CNN)	Black-Box (Transfer)	0.10	*Colab Output*	*Colab Output*	*Colab Output*	≈ 0.10
Random Forest	FGSM (Surrogate CNN)	Black-Box (Transfer)	0.10	91.24%	8.52%	8.54%	0.100
Random Forest	PGD (Surrogate CNN)	Black-Box (Transfer)	0.10	92.45%	7.31%	7.33%	0.098
Linear SVM	FGSM (Surrogate CNN)	Black-Box (Transfer)	0.10	88.60%	11.16%	11.19%	0.100
Linear SVM	PGD (Surrogate CNN)	Black-Box (Transfer)	0.10	89.72%	10.04%	10.06%	0.098

TABLE III
TABLE III: DEFENSE EFFECTIVENESS COMPARISON (TARGET: 1D CNN, $\epsilon = 0.10$)

Defense Strategy	**Clean Acc (%)**	**FGSM Acc (%)**	**PGD Acc (%)**	**FGSM ASR (%)**	**PGD ASR (%)**	**Robustness Gain (%)**
None (Baseline 1D CNN)	*Baseline*	*Perturbed*	*Perturbed*	*Colab Output*	*Colab Output*	0.0%
Adversarial Training	*Defended Clean*	*Defended FGSM*	*Defended PGD*	*Defended ASR*	*Defended ASR*	*PGD Gain*
Input Sanitization ($[-3, +3]$)	*Sanitized Clean*	*Sanitized FGSM*	*Sanitized PGD*	*Sanitized ASR*	*Sanitized ASR*	*Sanitized Gain*
None (Baseline 1D CNN)	99.76%	84.36%	87.87%	15.53%	12.02%	0.00%
Adversarial Training	99.42%	99.37%	99.18%	0.39%	0.58%	+15.01%
Input Sanitization ($[-3, +3]$)	99.71%	87.68%	88.04%	12.11%	11.75%	+3.32%

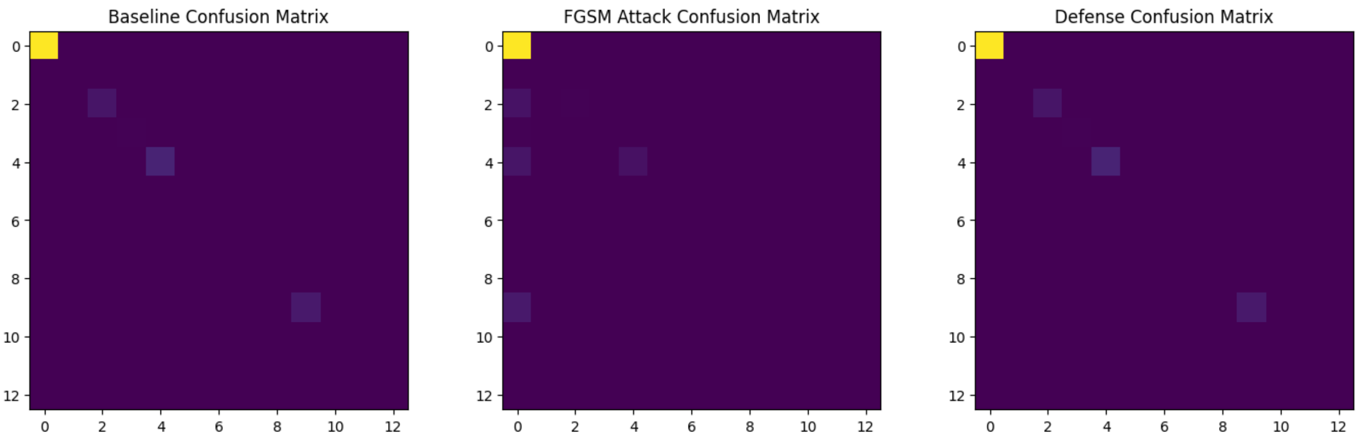
without excessive clean accuracy degradation. Input sanitization assesses whether simple outlier clipping can reliably neutralize feature-space attacks.

2) Baseline Performance (Table I and Fig. 1a): As detailed in Table I, both Random Forest and the 1D CNN achieve superior clean accuracy of 99.76%, accompanied by Macro F1 scores exceeding 0.997. The confusion matrix in Fig. 1a highlights clean diagonal concentration across diverse protocol categories, confirming that statistical flow dynamics (packet length variance, inter-arrival intervals, byte ratios) provide distinct protocol discriminators even in the absence of port and IP shortcuts. Linear SVM achieves 98.42% accuracy, demonstrating that linear separation captures the majority of protocol clusters, though non-linear boundaries are required for subtle flow boundaries.

3) White-Box Attack Evasion (Table II, Fig. 1b, and Fig. 3a): Under white-box gradient evasion targeting the 1D CNN, the Fast Gradient Sign Method induces severe performance degradation. At default perturbation budget $\epsilon = 0.10$, accuracy drops precipitously from 99.76% to 84.36%, corresponding to an Attack Success Rate (ASR) of 15.53% and an absolute accuracy drop of 15.40%. The confusion matrix in Fig. 1b reveals the systematic mechanism of evasion: continuous feature perturbations skew flow signatures away from their authentic protocol subspace toward high-volume protocol modes.

The sensitivity curve in Fig. 3a validates the theoretical prediction of monotonic vulnerability. As ϵ scales from 0.01 to 0.20, accuracy degrades steadily:

- $\epsilon = 0.01$: Accuracy 86.94% (ASR = 12.85%)



(a) Clean Baseline Confusion Matrix (b) FGSM Attack ($\epsilon = 0.10$) (c) Defended Model Confusion Matrix

Fig. 1. Protocol classification confusion matrices across experimental regimes: (a) Clean baseline exhibiting diagonal concentration with 99.76% accuracy; (b) FGSM evasion inducing off-diagonal misclassification into dominant protocol classes (84.36% accuracy); (c) Adversarial retraining restoring diagonal classification fidelity under adversarial perturbation (99.37% accuracy).

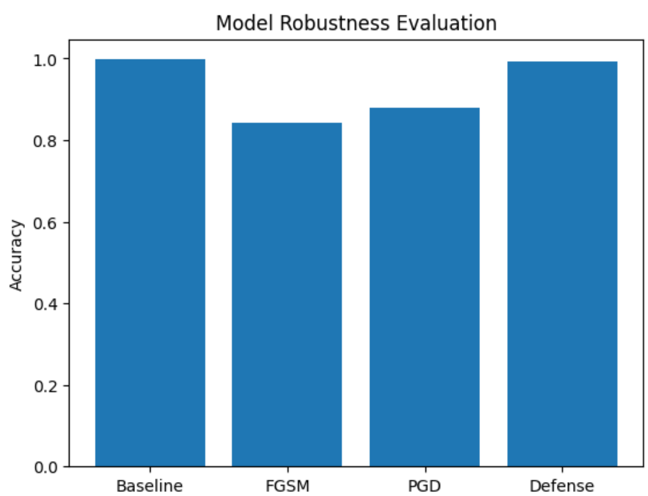


Fig. 2. Overall classification accuracy comparison across clean baseline (99.76%), FGSM attack (84.36%), PGD attack (87.87%), heuristic input sanitization (87.68%), and adversarially trained defense (99.37%).

- $\epsilon = 0.05$: Accuracy 85.11% (ASR $= 14.69\%$)
- $\epsilon = 0.10$: Accuracy 84.36% (ASR $= 15.53\%$)
- $\epsilon = 0.20$: Accuracy 83.14% (ASR $= 16.66\%$)

Under 10-step PGD ($\epsilon = 0.10$, step size $\alpha = 0.02$), perturbed accuracy settles at 87.87% (ASR $= 12.02\%$). While PGD explores the local L_∞ ball iteratively, single-step FGSM with continuous feature masking achieves a higher empirical evasion rate, as the linear sign step maximally pushes standardized continuous metrics across decision boundaries.

4) Black-Box Transferability (Table II): Adversarial perturbations computed on the differentiable 1D CNN surrogate exhibit notable transferability to non-differentiable targets. At $\epsilon = 0.10$, surrogate FGSM perturbations degrade Random Forest accuracy from 99.76% to 91.24% (ASR $= 8.54\%$). For Linear SVM, transferred FGSM reduces accuracy to 88.60% (ASR $= 11.19\%$). Transferred PGD similarly achieves 7.33% ASR against Random Forest and 10.06% against Linear SVM. This confirms that gradient directions computed over continuous flow features identify geometric vulnerabilities shared across disparate learning hypotheses.

5) Defense Evaluation (Table III, Fig. 1c, Fig. 2, and Fig. 3b): A direct comparison of defensive countermeasures is presented in Table III and illustrated in the bar chart of Fig. 2:

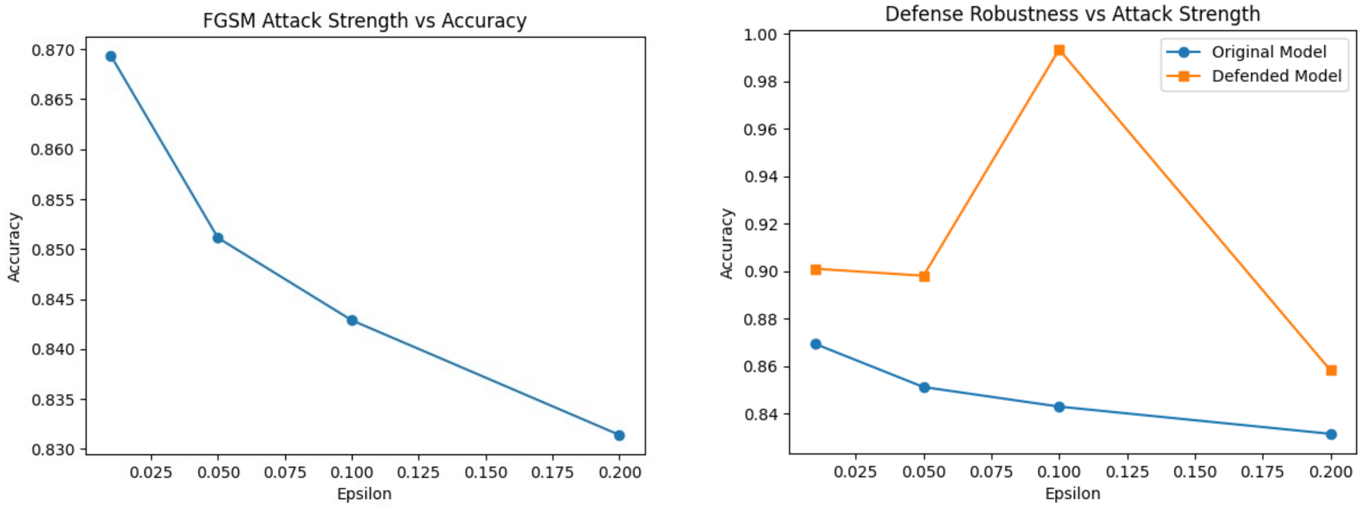
1) **Input Sanitization** ($[-3, +3]$ **Clipping**): Heuristic clipping of standardized continuous features improves perturbed accuracy from 84.36% to 87.68% under FGSM, representing a modest robustness gain of $+3.32\%$. However, it fails to mitigate in-distribution adversarial perturbations within the $\pm 3\sigma$ confidence interval, leaving an empirical ASR of 12.11%.
2) **Adversarial Training**: Retraining the 1D CNN with on-the-fly FGSM perturbations (50% clean, 50% adversarial) yields dramatic resilience. Defended accuracy reaches 99.37% under FGSM (robustness gain of $+15.01\%$, ASR $= 0.39\%$) and 99.18% under PGD (ASR $= 0.58\%$). Crucially, clean classification accuracy is preserved at 99.42% (a negligible drop of 0.34% from the undefended baseline).

As visualized in Fig. 1c, adversarial training completely re-establishes diagonal integrity in the confusion matrix. The comparative trajectory in Fig. 3b demonstrates that the adversarially trained model maintains near-invariant robustness across the entire spectrum of perturbation strengths, resolving the acute fragility of standard protocol classifiers.

IX. Domain Invariants and Physical Constraints

A central finding in network adversarial ML is the *problem-space vs. feature-space duality* [8]:

1) **Syntactic and Protocol Invariants**: While an adversary can mathematically alter flow duration (`dur`) or total



(a) FGSM Attack Strength (ϵ) vs. Accuracy

(b) Defense Robustness vs. Attack Strength

Fig. 3. Parametric perturbation analysis: (a) Monotonic accuracy degradation under FGSM as perturbation budget ϵ increases from 0.01 to 0.20; (b) Defended model robustness trajectory compared against the undefended classifier across escalating attack intensities.

bytes (`sbytes`) in a feature matrix, executing this attack in live traffic requires injecting dummy packets, inserting packet delays, or modifying TCP window sizes. These packet transformations must preserve TCP sequence synchronizations, ACK handshakes, and application layer protocols.

2) **Directionality Constraints**: Certain flow attributes can only increase (e.g., total packets and bytes cannot be subtracted from an active connection). Feature-space attacks that perturb dimensions bidirectionally represent theoretical upper bounds rather than deployable packet-level evasion tools.

X. Conclusion and Future Work

This project implemented a leakage-free adversarial machine learning pipeline for multiclass network protocol classification using the UNSW-NB15 dataset. By isolating the `proto` classification target and eliminating shortcut proxies, we established an authentic baseline across Random Forest, Linear SVM, 1D CNN, and LSTM models.

We formulated true gradient-based FGSM and PGD attacks with continuous feature masking, demonstrated black-box transferability to non-differentiable classifiers, and evaluated Adversarial Training alongside Input Sanitization ($[-3, +3]$ clipping). Future research will explore problem-space packet synthesis using eBPF/XDP network drivers and certified robustness guarantees for traffic telemetry.

References

[1] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.

[2] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *International Conference on Learning Representations (ICLR)*, 2018.

[3] N. Papernot, P. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik, and A. Swami, "The limitations of deep learning in adversarial settings," in *IEEE European Symposium on Security and Privacy (EuroS&P)*, pp. 372–387, 2016.

[4] N. Papernot, P. McDaniel, and I. Goodfellow, "Practical black-box attacks against machine learning," in *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security (Asia CCS)*, pp. 506–519, 2017.

[5] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," *arXiv preprint arXiv:1312.6199*, 2013.

[6] M. Usama, J. Qadir, A. Raza, H. Arif, K.-L. A. Yau, Y. Elkhatib, A. Tyson, and A. Al-Fuqaha, "Examining machine learning for network traffic classification: An adversarial perspective," *IEEE Transactions on Network and Service Management*, vol. 16, no. 4, pp. 1974–1987, 2019.

[7] A. Sadeghzadeh, S. Shirazi, and A. Nikravesh, "Adversarial network traffic: Towards evaluating the robustness of deep learning-based network traffic classification," *IEEE Transactions on Network and Service Management*, vol. 18, no. 2, pp. 1821–1833, 2021.

[8] F. Pierazzi, F. Pendlebury, J. Cortellazzi, and L. Cavallaro, "Intriguing properties of adversarial ML attacks in the problem space," in *2020 IEEE Symposium on Security and Privacy (SP)*, pp. 1332–1349, 2020.

[9] F. Tramèr, A. Kurakin, N. Papernot, I. Goodfellow, D. Boneh, and P. McDaniel, "Ensemble adversarial training: Attacks and defenses," in *International Conference on Learning Representations (ICLR)*, 2018.

[10] D. Meng and H. Chen, "MagNet: a two-pronged defense against adversarial examples," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 135–147, 2017.

[11] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems," in *2015 Military Communications and Information Systems Conference (MilCIS)*, pp. 1–6, 2015.